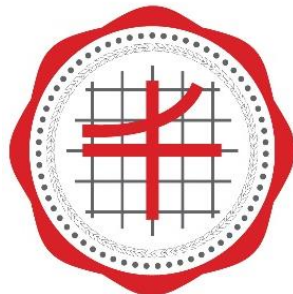


CPE 121 Mobile Application Developments

บทที่ 4

เมธอดและคอนสตรัคเตอร์ (Method and Constructor)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.

Department of Computer Engineering

เมธอด

เมธอด (Method) คือ กลุ่มคำสั่งที่ถูกกำหนดขึ้นเพื่อทำงานอย่างใดอย่างหนึ่งให้ได้ผลลัพธ์ตามต้องการ โดยจะต้องเป็นสมาชิกของคลาส เปรียบได้กับโปรแกรมย่อย (Function) ในโปรแกรมแบบมีโครงสร้าง มีรูปแบบการใช้งานดังนี้

```
[modifier] return_type Method_name (parameter) {  
    [method_body]  
    return varValue; }
```

โดยที่

modifier	เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงเมธอด เช่น ถ้าเป็น public จะสามารถเรียกใช้จากที่ใดก็ได้ แต่ถ้าเป็น private จะเรียกใช้ได้เฉพาะคลาสเดียวกันเท่านั้น
return_type	เป็นส่วนที่บอกถึงการคืนค่าหรือไม่ และคืนค่าข้อมูลเป็นประเภทใด โดยที่หากไม่มีการคืนค่าจะใช้ void และ หากมีการคืนค่าจะต้องกำหนดเป็นประเภทของข้อมูลที่จะคืนออกมาไว้ด้วย
MethodName	เป็นชื่อเมธอด
parameter	เป็นตัวแปรที่ใช้ในการรับข้อมูล
varValue	เป็นค่าที่ต้องการส่งค่ากลับ

รูปแบบการเรียกใช้งานเมธอด

- เมธอดไม่มีการรับพารามิเตอร์และไม่มีการคืนค่า
- เมธอดมีการรับพารามิเตอร์แต่ไม่มีการคืนค่า
- เมธอดไม่มีการรับพารามิเตอร์แต่มีการคืนค่า
- เมธอดมีการรับพารามิเตอร์และมีการคืนค่า



เมธอดไม่มีการรับพารามิเตอร์และไม่มีการคืนค่า

การเรียกใช้เมธอดแบบที่ไม่มีการรับพารามิเตอร์และไม่มีการคืนค่ามีรูปแบบดังนี้

```
ObjectName.MethodName ();
```

โดยที่

ObjectName

เป็นชื่อออบเจกต์

MethodName

เป็นชื่อเมธอดที่ต้องการใช้งาน

เมธอดไม่มีการรับพารามิเตอร์และไม่มีการคืนค่า

```
public class Wage_NoParam_Void {
    public static void main(String[] args){
        wage emp_wage = new wage();
        emp_wage.h = 50; emp_wage.r = 100.0f;
        emp_wage.calWage();
    }
}
```

> run Wage_NoParam_Void

ค่าแรงรวม = 5500.0 บาท

```
class wage {
    public int h;
    public float r;
    public void calWage() {
        float total = (40*r)+(h-40)*(1.5f*r);
        System.out.println("ค่าแรงรวม = " + total + " บาท ");
    }
}
```



เมธอดมีการรับพารามิเตอร์แต่ไม่มีการคืนค่า

การเรียกใช้เมธอดที่มีการรับพารามิเตอร์แต่ไม่มีการคืนค่ามีรูปแบบดังนี้

```
ObjectName.MethodName ([argument]);
```

โดยที่

ObjectName

เป็นชื่อออบเจกต์

MethodName

เป็นชื่อเมธอดที่ต้องการใช้งาน

argument

เป็นค่าข้อมูลที่ต้องการส่งผ่านไปให้เมธอด



เมธอดมีการรับพารามิเตอร์แต่ไม่มีการคืนค่า

```
public class Wage_Param_Void {
    public static void main(String[] args){
        wage emp_wage = new wage();
        emp_wage.calWage(50,200.0f);
    }
}
```

> run Wage_Param_Void

ค่าแรงรวม = 11000.0 บาท

```
class wage {
    public void calWage(int h, float r) {
        float total = (40*r)+(h-40)*(1.5f*r);
        System.out.println("ค่าแรงรวม = " + total + " บาท");
    }
}
```



เมธอดไม่มีการรับพารามิเตอร์แต่มีการคืนค่า

การเรียกใช้เมธอดที่ไม่มีการรับพารามิเตอร์แต่มีการคืนค่ามีรูปแบบดังนี้

```
dataType MethodValue = ObjectName.MethodName ();
```

โดยที่

ObjectName	เป็นชื่อออบเจกต์
MethodName	เป็นชื่อเมธอดที่ต้องการใช้งาน
dataType	เป็นชนิดข้อมูลของเมธอดที่มีการคืนค่า
MethodValue	เป็นตัวแปรที่ใช้เก็บค่าที่ได้จากการคืนค่าของเมธอด



เมธอดไม่มีการรับพารามิเตอร์แต่มีการคืนค่า

```
public class Wage_NoParam_Return {
    public static void main(String[] args){
        wage emp_wage = new wage();
        emp_wage.h = 50; emp_wage.r = 300.0f;
        float total = emp_wage.calWage();
        System.out.println("ค่าแรงรวม = " + total + " บาท ");
    }
}
```

> run Wage_NoParam_Return

ค่าแรงรวม = 16500.0 บาท

```
class wage {
    public int h;
    public float r;
    public float calWage() {
        return (40*r)+(h-40)*(1.5f*r);
    }
}
```



เมธอดที่มีการรับพารามิเตอร์และมีการคืนค่า

การเรียกใช้เมธอดที่มีการรับพารามิเตอร์และมีการคืนค่ามีรูปแบบดังนี้

```
dataType MethodValue = ObjectName.MethodName ([argument]);
```

โดยที่

ObjectName	เป็นชื่อออบเจกต์
MethodName	เป็นชื่อเมธอดที่ต้องการใช้งาน
argument	เป็นค่าข้อมูลที่ต้องการส่งผ่านไปให้เมธอด
dataType	เป็นชนิดข้อมูลของเมธอดที่มีการคืนค่า
MethodValue	เป็นตัวแปรที่ใช้เก็บค่าที่ได้จากการคืนค่าของเมธอด



เมธอดมีการรับพารามิเตอร์และมีการคืนค่า

```
public class Wage_Param_Return {
    public static void main(String[] args){
        wage emp_wage = new wage();
        float total = emp_wage.calWage(50,400.0f);
        System.out.println("ค่าแรงรวม = " + total + " บาท");
    }
}
```

> run Wage_Param_Return

ค่าแรงรวม = 22000.0 บาท

```
class wage {
    public float calWage(int h, float r) {
        return (40*r)+(h-40)*(1.5f*r);
    }
}
```



การเรียกใช้งานเมธอด

การเรียกใช้งานเมธอดที่กำหนดขึ้นในคลาสสามารถทำได้ 2 รูปแบบ คือ

1. เรียกใช้งานจากคลาสที่ต่างกัน ต้องสร้างออบเจกต์จากคลาสที่มีเมธอดที่เราต้องการเรียกใช้งาน และเรียกใช้เมธอดผ่านออบเจกต์ที่สร้างขึ้น ซึ่งเมธอดดังกล่าวต้องไม่มี **access modifier** เป็นแบบ **private**
2. เรียกใช้งานภายในคลาสเดียวกัน ซึ่งสามารถเรียกผ่านชื่อเมธอดได้โดยไม่ต้องสร้างออบเจกต์ เมื่อเมธอดนั้นไม่เป็นเมธอดแบบ **static**



โปรแกรมคำนวณค่าแรงเรียกใช้เมธอดจากคลาส

```
public class WageMethod {
    public static void main(String[] args){
        wage emp_wage = new wage();
        float total = emp_wage.calWage(50,500.0f);
        System.out.printf("%s%,10.2f %s", "ค่าแรงรวม =", total, "บาท\n");
    }
}
```

> run WageMethod

ค่าแรงรวม = 27,500.00 บาท

```
class wage {
    public float calWage(int h, float r) {
        return (40*r) + calOt(h,r);
    }
    private float calOt(int h, float r){
        return (h-40) * (1.5f*r);
    }
}
```



อาร์กิวเมนต์และพารามิเตอร์

อาร์กิวเมนต์ (Argument) คือ ตัวแปรที่ส่งไปให้เมธอดพร้อมกับการเรียกใช้เมธอด ในกรณีมีจำนวนมากกว่าหนึ่งค่าให้คั่นด้วยเครื่องหมาย **","**

พารามิเตอร์ (Parameter) คือ ตัวแปรที่ทำหน้าที่รับค่าอาร์กิวเมนต์ที่ส่งมาใช้งานในเมธอด ในกรณีมีจำนวนมากกว่าหนึ่งค่าให้คั่นด้วยเครื่องหมาย **","**

- ตัวแปรที่เป็นอาร์กิวเมนต์ต้องมีจำนวนเท่ากับตัวแปรที่เป็นพารามิเตอร์
- ชนิดข้อมูลของอาร์กิวเมนต์ กับพารามิเตอร์ ในแต่ละตำแหน่งจะต้องสอดคล้องกัน แต่ไม่จำเป็นที่จะต้องเป็นข้อมูลชนิดเดียวกัน กล่าวคือ อนุญาตให้เป็นชนิดข้อมูลที่สามารถ **แปลงข้อมูลโดยอัตโนมัติได้ (Implicit Type Conversion)**
 - ส่งอาร์กิวเมนต์ที่มีชนิด **float** ให้กับเมธอดที่มีพารามิเตอร์ชนิดข้อมูลเป็น **double** ได้
 - ส่งอาร์กิวเมนต์ที่มีชนิด **int** ให้กับเมธอดที่มีพารามิเตอร์ชนิดข้อมูลเป็น **double** ได้



โปรแกรมคำนวณค่าแรง

- พารามิเตอร์และอาร์กิวเมนต์ชนิดข้อมูลสอดคล้องกัน

```
public class Wage_Arg_Type {
    public static void main(String[] args){
        wage x = new wage();
        int hr = 50; float rate = 600.0f;
        double total = x.calWage(hr, rate);
        System.out.printf("%s%,10.2f %s", "ค่าแรงรวม =", total, "บาท\n");
    }
}
```

> run Wage_Arg_Type

ค่าแรงรวม = 33,000.00 บาท

```
class wage {
    public double calWage(float h, double r) {
        return (40*r)+(h-40)*(1.5f*r);
    }
}
```



การส่งค่าข้อมูล

- การส่งชนิดข้อมูลพื้นฐานจะเป็นการส่งค่าข้อมูล (pass by value) ไปให้กับเมธอดที่เรียกใช้ หากมีการเปลี่ยนแปลงค่าของพารามิเตอร์ภายในเมธอดที่เราเรียกใช้ จะไม่ทำให้ค่าของอาร์กิวเมนต์เปลี่ยนตาม
- การส่งชนิดข้อมูลแบบอ้างอิงซึ่งใช้กับข้อมูลประเภทออบเจกต์ จะเป็นการส่งค่าตำแหน่งอ้างอิง (pass by reference) ไปให้กับเมธอดที่เรียกใช้ ดังนั้น การเปลี่ยนแปลงค่าพารามิเตอร์ภายในเมธอดที่เรียกใช้ จะมีผลทำให้ค่าของอาร์กิวเมนต์ที่ส่งไปเปลี่ยนตาม
- คุณลักษณะเช่นนี้สามารถนำไปประยุกต์ใช้กับกรณีที่ต้องการเปลี่ยนแปลงค่าของอาร์กิวเมนต์ที่ส่งไปภายใต้การทำงานของเมธอด 1 เมธอด เช่น
 - การคำนวณการแลกเหรียญที่ต้องการผลเป็นจำนวนเหรียญ 10 บาท, เหรียญ 5 บาท, เหรียญ 2 บาท และเหรียญ 1 บาท ซึ่งผลลัพธ์ที่ต้องการมีทั้งหมด 4 ค่า หรือ
 - การคำนวณหาคะแนนสูงสุด, คะแนนต่ำสุด และคะแนนเฉลี่ย เป็นต้น



โปรแกรมคำนวณแลกเหรียญ

```
public class Coin_By_Ref {
    public static void main(String args[]) {
        int money = 29;
        coins coin = new coins();
        ExchangeCoin coinData = new ExchangeCoin();
        coinData.findCoin(coin,money);
        System.out.println("เงิน " + money + " บาท แลกได้ " + " เหรียญสิบ "
            + coin.ten + " เหรียญ เหรียญห้า " + coin.five + " เหรียญ เหรียญบาท "
            + coin.one + " เหรียญ ");
    }
}
```

```
class coins {
    int ten,five,one;
}
class ExchangeCoin {
    public void findCoin(coins c, int m) {
        c.ten = m/10;  c.five = m%10/5;  c.one = m%10%5;
    }
}
```

> run Coin_By_Ref

เงิน 29 บาท แลกได้เหรียญสิบ 2 เหรียญ
เหรียญห้า 1 เหรียญ เหรียญบาท 4 เหรียญ

ประเภทของเมธอด

- Instance Method
- Static Method
- Constructor method
- Overloading Method
- Overriding Method



Instance Method

Instance Method เป็นเมธอดที่เรียกผ่านออบเจกต์ที่สร้างจากคลาสด้วยตัวดำเนินการ **new** ซึ่งเมธอดที่สร้างเพื่อการใช้งานในตัวอย่างที่ผ่านมา จัดเป็นเมธอดประเภท instance method

```
public class instMethod {  
    public static void main(String[] args) {  
        System.out.println("Bonus = " + new Bonus().calBonus(50000) + " baht");  
    }  
}
```

```
class Bonus {  
    public float calBonus(float s){  
        return 0.05f*s;  
    }  
}
```



Static Method

Static Method เป็นเมธอดที่เรียกใช้ได้โดยไม่ต้องสร้างออบเจกต์ สามารถเรียกใช้เมธอดประเภทนี้ผ่านชื่อคลาสได้เลย แต่จะต้องเรียกใช้จากเมธอดประเภท static method เหมือนกัน

```
public class statMethod {  
    public static void main(String[] args) {  
        System.out.println("Bonus = " + Bonus.calBonus(40000) + " baht");  
    }  
}
```

```
class Bonus {  
    public static float calBonus(float s){  
        return 0.05f*s;  
    }  
}
```

โปรแกรมคำนวณเงินภาษีด้วย static method

```
public class myProduct {
    public static void main(String[] args){
        float money = 5000;
        float total_vat = tax.calVat(money);
        float total_fuel = tax.calFuel(money);
        System.out.println("สินค้าราคา " + money +
            " บาท คิดภาษีมูลค่าเพิ่ม 7% เป็นเงิน " + total_vat +
            " บาท\nคิดภาษีน้ำมัน 3% เป็นเงิน " + total_fuel +
            " บาท รวมเป็นเงินภาษี " + (total_vat+total_fuel) +
            " บาท");
    }
}
```

```
public class tax {
    public static float calVat(float m) {
        return 0.07f*m;
    }
    public static float calFuel(float m) {
        return 0.03f*m;
    }
}
```

> run myProduct

สินค้าราคา 5000.0 บาท คิดภาษีมูลค่าเพิ่ม 7% เป็นเงิน 350.0 บาท
 คิดภาษีน้ำมัน 3% เป็นเงิน 150.0 บาท รวมเป็นเงินภาษี 500.0 บาท

Constructor method

Constructor method เป็นเมธอดที่มีการกำหนดชื่อเมธอดเป็นชื่อเดียวกันกับชื่อคลาส เพื่อกำหนดให้เมธอดดังกล่าวทำงานเป็นเมธอดแรกเมื่อเรียกใช้งานคลาส ซึ่งการทำงานของ Constructor method จะเหมาะสำหรับการกำหนดค่าเริ่มต้นให้กับออบเจกต์

```
public class constMethod {
    public static void main(String[] args) {
        System.out.println("Bonus = " +
            new Bonus().myBonus + " baht");
        System.out.println("Bonus = " +
            new Bonus().calBonus(30000) + " baht");
    }
}
```

> run constMethod

Bonus = 0.0 baht

Bonus = 1500.0 baht

```
public class Bonus {
    public float myBonus;
    public Bonus() {
        myBonus = 0;
    }
    public float calBonus(float s){
        myBonus = 0.05f*s;
        return myBonus;
    }
}
```

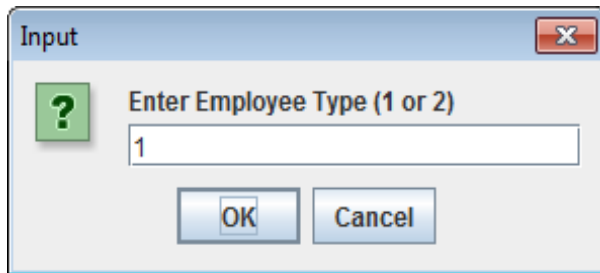


Overloading Method

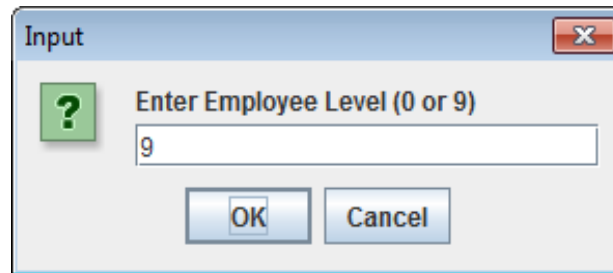
Overloading Method เป็นเมธอดที่มีคุณลักษณะที่มีได้หลายรูปแบบ (Polymorphism) โดยใช้ชื่อเมธอดเดียวกันมากกว่า 1 เมธอด เพื่อทำงานในแบบเดียวกัน สิ่งที่แตกต่างกันคือ ชนิดข้อมูลของพารามิเตอร์ หรือมีจำนวนและชนิดข้อมูลของอาร์กิวเมนต์ที่ใช้ในการรับข้อมูลแตกต่างกัน

โปรแกรมตรวจสอบเงินเดือน โดยการทำ overload เมธอด setsalary()

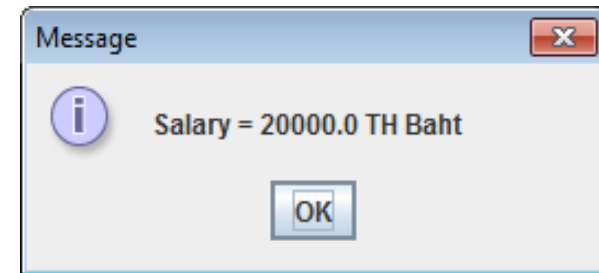
- กรณีเลือกประเภทพนักงานเป็น 1 และระดับพนักงานเป็น 9



Input dialog box titled "Enter Employee Type (1 or 2)". The input field contains the value "1". There are "OK" and "Cancel" buttons at the bottom.

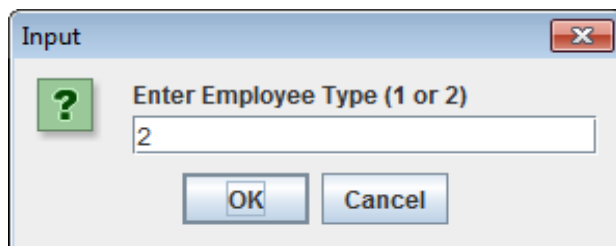


Input dialog box titled "Enter Employee Level (0 or 9)". The input field contains the value "9". There are "OK" and "Cancel" buttons at the bottom.

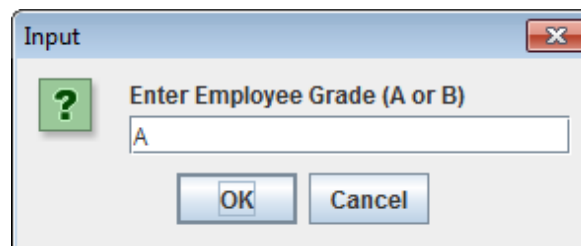


Message dialog box titled "Salary = 20000.0 TH Baht". There is an "OK" button at the bottom.

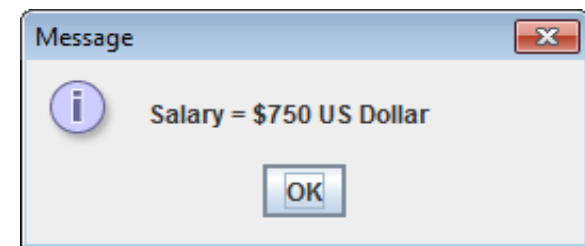
- กรณีเลือกประเภทพนักงานเป็น 2 และเกรดพนักงานเป็น A



Input dialog box titled "Enter Employee Type (1 or 2)". The input field contains the value "2". There are "OK" and "Cancel" buttons at the bottom.



Input dialog box titled "Enter Employee Grade (A or B)". The input field contains the value "A". There are "OK" and "Cancel" buttons at the bottom.



Message dialog box titled "Salary = \$750 US Dollar". There is an "OK" button at the bottom.

Overloading Method

```
public class EMPLOYEES {  
    public static void main(String[] args) {  
        String data;  
        data=JOptionPane.showInputDialog("Enter Employee Type (1 or 2) ");  
        int n = new Integer(data);  
        employee_salary emp = new employee_salary();  
        switch (n) {  
            case 1:  
                data=JOptionPane.showInputDialog("Enter Employee Level (0 or 9) ");  
                float s1 = emp.setsalary(new Integer(data));  
                JOptionPane.showMessageDialog(null,"Salary = "+ s1 + " TH Baht");    break;  
            case 2:  
                data=JOptionPane.showInputDialog("Enter Employee Grade (A or B) ");  
                String s2 = emp.setsalary(data);  
                JOptionPane.showMessageDialog(null,"Salary = "+s2 + " US Dollar");    break;  
            default :  
                JOptionPane.showMessageDialog(null,"Invalid Employee Type");  
        } }  
    }
```


Overloading Method

```
import javax.swing.JOptionPane;
class employee_salary {
    float setsalary(int t1) {
        float salary=0;
        if (t1==0)
            salary = 10000;
        if (t1==9)
            salary = 20000;
        return salary;
    }
    String setsalary(String t2) {
        String salary="";
        if (t2.equals("A"))
            salary = "$750";
        if (t2.equals("B"))
            salary = "$500";
        return salary;
    }
}
```



Overriding Method

Overriding Method เป็นเมธอดที่มีคุณลักษณะของ Polymorphism อีกรูปแบบหนึ่ง ที่คลาสลูกสามารถเขียนทับเมธอดของคลาสแม่ได้ โดยที่จำนวนอาร์กิวเมนต์, ชนิดข้อมูลของอาร์กิวเมนต์ และชนิดของข้อมูลที่คืนค่าจะต้องเหมือนในคลาสแม่เสมอ

```
public class OverridingMethod {
    public static void main(String[] args) {
        System.out.println("triangle area = " +
            new tri().calArea(2,2));
        System.out.println("rectangle area = " +
            new rec().calArea(2,2));
    }
}
```

```
class shape {
    public float calArea(float x, float y) {
        return 0;
    }
}
```

```
class tri extends shape {
    @Override
    public float calArea(float x, float y) {
        return 0.5f*x*y;
    }
}

class rec extends shape {
    @Override
    public float calArea(float x, float y) {
        return x*y;
    }
}
```

> run OverridingMethod

triangle area = 2.0

rectangle area = 4.0

คอนสตรัคเตอร์

คอนสตรัคเตอร์ (Constructor) คือ เป็นเมธอดประเภทหนึ่งที่มีชื่อเดียวกับชื่อคลาส เมื่อมีการสร้างออบเจกต์ด้วยคำสั่ง **new** เมธอดนี้จะถูกเรียกใช้งานโดยอัตโนมัติ เป็นเมธอดที่ไม่มีการคืนค่าและไม่ต้องระบุคีย์เวิร์ด **void** ถ้าในโปรแกรมไม่มีคอนสตรัคเตอร์ คอมไพเลอร์ภาษา Java จะใส่ คอนสตรัคเตอร์ แบบ **default** ให้กับโปรแกรมโดยอัตโนมัติ คอนสตรัคเตอร์ แบบ **default** จะเป็น คอนสตรัคเตอร์ ที่ไม่มีคำสั่งใดๆ อยู่ภายใน

```
public class Calculate {  
    public double dblSquareArea;  
    public int intSquareArea;  
  
    Calculate () {  
        dblSquareArea = 0;;  
        intSquareArea = 0;  
    }  
}
```

การสร้างคอนสตรัคเตอร์

เพื่อกำหนดค่าเริ่มต้นให้กับแอตทริบิวต์หรือเมธอดต่าง ๆ ก่อนเริ่มใช้งาน คอนสตรัคเตอร์ สามารถรับพารามิเตอร์ได้เหมือนกับเมธอดมีรูปแบบการใช้งาน ดังนี้

```
[modifier] ClassName ([parameter]) {
    [statements]
}
```

โดยที่

ClassName

modifier

parameter

เป็นชื่อคลาส

เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงเมธอด

เป็นตัวแปรที่ใช้รับข้อมูลอาร์กิวเมนต์จากคลาสที่เรียกใช้งาน



การใช้งานคอนสตรัคเตอร์

ในการเรียกใช้งานคอนสตรัคเตอร์ ก็คือการประกาศออบเจกต์นั่นเอง เนื่องจากคอนสตรัคเตอร์ จะมีการทำงานโดยอัตโนมัติเมื่อสร้างออบเจกต์ใหม่ ซึ่งรูปแบบการสร้างออบเจกต์ดังนี้

```
ClassName objName = new ClassName ([argument]);
```

โดยที่

ClassName	เป็นชื่อคลาส
objName	เป็นชื่อออบเจกต์
argument	เป็นค่าอาร์กิวเมนต์ที่จะส่งไปให้กับคอนสตรัคเตอร์

- พารามิเตอร์ของคอนสตรัคเตอร์จะต้องมีจำนวนและชนิดข้อมูลสอดคล้องกับอาร์กิวเมนต์ที่ส่งมาตอนประกาศออบเจกต์ด้วยคำสั่ง **new**



โปรแกรมการใช้งาน Default Constructor

```
public class DefaultConstructorTest {  
    public static void main(String args[]) {  
        String emp_id = "CPE222";  
        Employee0 emp = new Employee0();  
        System.out.println("== Default Constructor ==");  
        System.out.println("Employee ID = " + emp_id + "\nSalary = "+  
emp.emp_salary);  
        System.out.println("=====");  
    }  
}
```

```
class Employee0 {  
    public double emp_salary;  
}
```



โปรแกรมการใช้งาน Constructor

```
public class ConstructTest {  
    public static void main(String args[]) {  
        String emp_id = "CPE222";  
        Employee1 emp = new Employee1();  
        System.out.println("== Constructor Test===");  
        System.out.println("Employee ID = " + emp_id + "\nSalary = " + emp.emp_salary);  
        System.out.println("=====");  
    }  
}
```

```
class Employee1 {  
    public double emp_salary;  
    public Employee1 () {  
        emp_salary = 10000;  
    }  
}
```



โอเวอร์โหลดคอนสตรัคเตอร์

โอเวอร์โหลดคอนสตรัคเตอร์ (**Overload Constructor**) หมายถึง คอนสตรัคเตอร์ ที่มีมากกว่าหนึ่งตัวในคลาสเดียวกัน มีหลักการเดียวกับ **Overloading Method** โดยที่จำนวนหรือชนิดข้อมูลของอาร์กิวเมนต์ที่ส่งไปในแต่ละคอนสตรัคเตอร์ต้องแตกต่างกัน คอมไพเลอร์จะเรียกใช้คอนสตรัคเตอร์ ตัวใดให้ทำงานนั้นจะพิจารณาจากจำนวนหรือชนิดข้อมูลของอาร์กิวเมนต์ที่สอดคล้องกัน ทำให้กำหนดค่าเริ่มต้นให้กับออบเจกต์ได้หลายรูปแบบ



โปรแกรมการใช้งานโอเวอร์โหลดคอนสตรัคเตอร์

```
public class OverloadTest {  
    public static void main(String args[]) {  
        double data = 20000;  
        Employee2 emp1 = new Employee2();  
        Employee2 emp2 = new Employee2(data);  
        System.out.println("== Overload Test==");  
        System.out.println("Employee ID = " + emp1.emp_id + "\nSalary = " + emp2.emp_salary);  
        System.out.println("=====");  
    }  
}
```

```
class Employee2 {  
    String emp_id;  
    double emp_salary;  
    public Employee2 () {  
        emp_id = "CPE222"; emp_salary = 20000;  
    }  
    public Employee2 (double s) {  
        emp_salary = s; emp_id = "CPE222"  
    }  
}
```

การใช้คีย์เวิร์ด **this()**

this() เป็นคีย์เวิร์ดที่ใช้เรียก คอนสตรัคเตอร์ที่อยู่ภายในคลาสเดียวกันเป็น **คำสั่งแรกสุด** ของคอนสตรัคเตอร์ที่เป็นตัวเรียกใช้งาน โดยมีรูปแบบเหมือนกับการเรียกใช้ด้วยคำสั่ง **new** คือ สามารถส่งชนิดข้อมูลของอาร์กิวเมนต์ที่ต้องการได้ เพื่อให้สามารถเรียกใช้คอนสตรัคเตอร์ที่สอดคล้องกันได้



โปรแกรมการใช้คีย์เวิร์ด **this()**

```
public class ThisTest {
    public static void main(String args[]) {
        double data = 30000;
        Employee3 emp = new Employee3(data);
        System.out.println("== This Test==");
        System.out.println("Employee ID = " + emp_id + "\nSalary = " + emp.emp_salary);
        System.out.println("=====");
    }
}
```

```
class Employee3 {
    String emp_id;
    double emp_salary;
    public Employee3 () {
        emp_id = "CPE222"; }
    public Employee3 (String n) {
        emp_id = n; }
    public Employee3 (double s) {
        this("CPESWU"); >> emp_id = "CPESWU";
        emp_salary = s; }
}
```

การใช้คีย์เวิร์ด **super()**

super() เป็นคีย์เวิร์ดสำหรับการเรียกใช้ คอนสตรัคเตอร์ในคลาสแม่ เนื่องจากคอนสตรัคเตอร์จะไม่มี การสืบทอดมาให้คลาสลูก คลาสลูกที่ต้องการเรียกใช้คอนสตรัคเตอร์ ในคลาสแม่ จึงใช้เมธอด **super()** เรียกใช้เป็นคำสั่งแรกสุดของ คอนสตรัคเตอร์และมีรูปแบบเหมือนกับการเรียกใช้ด้วยคำสั่ง **new**



โปรแกรมการใช้คีย์เวิร์ด **super()**

```
public class SuperTest {  
    public static void main(String args[]) {  
        double data = 40000;  
        EmployeeSale emp = new EmployeeSale(data);  
        System.out.println("== Super Test==");  
        System.out.println("Employee ID = " + emp_id + "\nSalary = " + emp.emp_salary);  
        System.out.println("=====");  
    }  
}
```



โปรแกรมการใช้คีย์เวิร์ด **super()**

```
class Employee4 {  
    String emp_id;  
    double emp_salary;  
    public Employee4 () {  
        emp_id = "CPE222";  
    }  
    public Employee4 (String n) {  
        emp_id = n;  
    }  
}
```

```
class EmployeeSale extends Employee4 {  
    public EmployeeSale(double s) {  
        super("CPESWU");  
        emp_salary = s;  
    }  
}
```

Unified Modeling Language

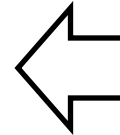
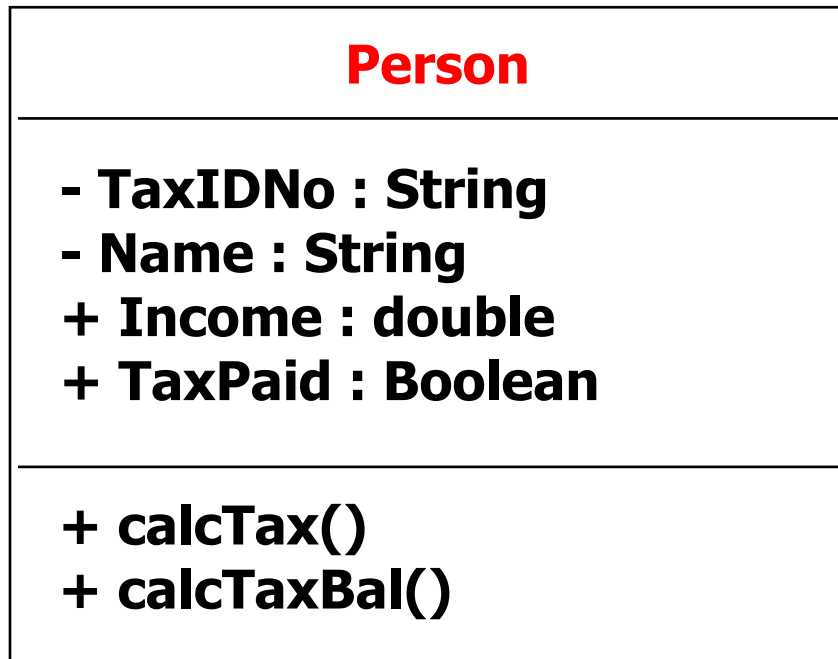
Unified Modeling Language (UML) เป็นภาษารูปภาพมาตรฐาน (Standard Modeling Language) สำหรับใช้ในการสร้างโมเดลเชิงวัตถุ โดย UML เป็นเสมือนพิมพ์เขียวที่แสดงภาพรวมของระบบทั้งหมด โดยจะแสดงในรูปแบบของแผนภาพ (Diagram) เพื่อให้เกิดความเข้าใจที่ตรงกันระหว่างผู้ออกแบบระบบ, โปรแกรมเมอร์ และผู้ใช้งาน

Class Diagram เป็นแผนภาพที่ใช้แสดง Class และความสัมพันธ์ระหว่าง Class ของระบบที่สนใจ (Problem Domain) เช่น ในระบบจัดซื้อ Class ที่เกี่ยวข้อง คือ ผู้ผลิต, พนักงานจัดซื้อ, ใบสั่งซื้อ, ใบเสนอราคา, ใบเสร็จรับเงิน เป็นต้น โดยที่สัญลักษณ์ของ Class Diagram ประกอบด้วย

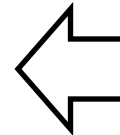
- **Class Name** คือ ชื่อของ Class
- **Attributes** คือ คุณลักษณะของ Class
- **Operations** หรือ Methods คือ กิจกรรมที่สามารถกระทำกับ Object นั้นๆ ได้



Class Diagram

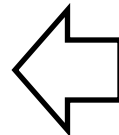


Class name



Attributes

attribute name : type



Operations

operation name(parameter : type)
: result type

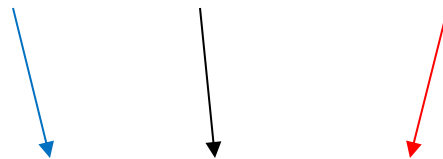


Class Diagram

▪ UML Syntax for **Attributes**

visibility name : **type**

(+, -, #) id : **String**



-visibility: public (+), protected (#), private (-)

-name : เป็น string

-type : ประเภทของข้อมูล



Class Diagram

▪ UML Syntax for **operations**

visibility name (**parameter list**) : **return-type-expression**

(+, -, #) assignAgent (a : Agent) : Boolean

-**visibility**: public (+), protected (#), private (-)

-name : string

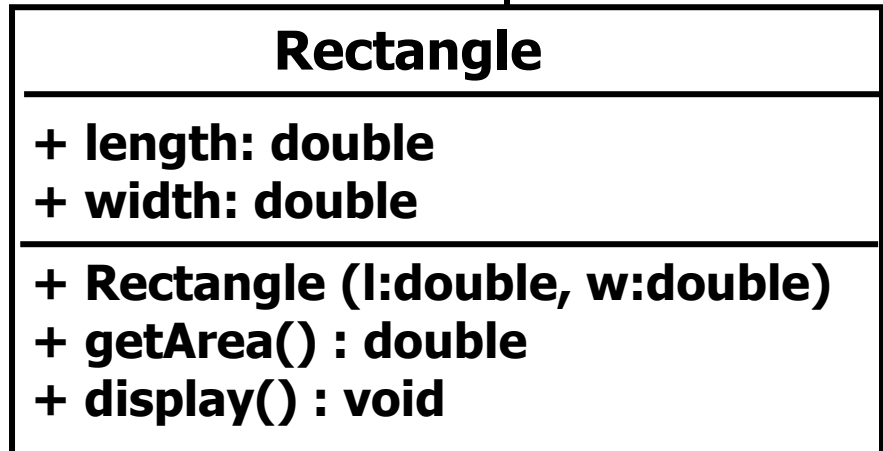
-**parameter list** : arguments

- **return-type-expression** : ประเภทของค่าที่ return



Class Diagram

```
public class Rectangle{  
    public double length;  
    public double width;  
    public Rectangle(double l, double w){  
        this.length = l;  
        this.width = w;  
    }  
    public double getArea(){  
        return length*width;  
    }  
    public void display(){  
        System.out.println("Length of this rectangle is " + length);  
        System.out.println("Width of this rectangle is " + width);  
        System.out.println("Area of this rectangle is " + getArea());  
    }  
}
```



การห่อหุ้ม

ลักษณะที่สำคัญของการเขียนโปรแกรมเชิงวัตถุ คือ ข้อมูลหรือคุณลักษณะบางอย่างที่สร้างขึ้นในคลาสสามารถปกป้องหรือถูกห่อหุ้ม(Encapsulation) อยู่ภายใน ไม่ให้ออบเจกต์สามารถเข้าใช้งานหรือเข้าไปแก้ไขข้อมูลได้ โดยในออบเจกต์หนึ่งๆ ควรที่จะมีบางส่วนที่ใช้ได้ภายในออบเจกต์เท่านั้น ออบเจกต์อื่นๆไม่ควรสามารถเข้ามาใช้งานได้ และมีบางส่วนที่อนุญาตให้ออบเจกต์อื่นๆเข้ามาใช้งานได้ โดยรูปแบบของการห่อหุ้มจะประกอบด้วย

- **public** ใช้นิยามตัวแปร, Method และ Class ใด ๆ เพื่อให้สามารถนำไปใช้กับ Class หรือโปรแกรมอื่น ๆ ได้
- **private** ใช้นิยามตัวแปรหรือ Method เพื่อให้เรียกใช้ได้เฉพาะภายใน Class ที่สร้างตัวแปร หรือ Method นั้น ๆ ขึ้นมาเท่านั้น
- **protected** ใช้นิยามตัวแปรหรือ Method ที่ใช้ได้เฉพาะ Class ที่สร้างขึ้นมาด้วยวิธีการสืบทอด (Inheritance) เท่านั้น โดยปกติจะใช้ Protected กับ Class ที่เป็น Class ต้นฉบับ (BaseClass)



การห่อหุ้ม

Rectangle1

+ length: double
+ width: double

+ Rectangle1()
+ Rectangle1 (l:double, w:double)
+ getArea() : double
+ display() : void

Rectangle2

- length: double
- width: double

+ Rectangle2()
+ Rectangle2 (l:double, w:double)
+ getLength() : double
+ getWidth() : double
+ getArea() : double
+ display() : void



การห่อหุ้ม

```
public class Rectangle1{
    public double length;
    public double width;
    public Rectangle1(){
        this.length = 1;
        this.width = 1;
    }
    public Rectangle1(double l, double w){
        this.length = l;
        this.width = w;
    }
    public double getArea(){
        return length*width;
    }
    public void display(){
        System.out.println("Area of this rectangle is " + getArea());
    }
}
```

Rectangle1
+ length: double + width: double
+ Rectangle1() + Rectangle1(l:double, w:double) + getArea() : double + display() : void

การห่อหุ้ม

```
import java.util.Scanner;
public class TestRectangle1 {
    public static void main(String[] args) {
        Rectangle1 TRect = new Rectangle1();
        TRect.display();
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter Length: ");
        double l = scan.nextDouble();
        System.out.println("Enter Width: ");
        double w = scan.nextDouble();
        Rectangle1 TRect1 = new Rectangle1(l,w);
        System.out.println("Length of this rectangle is " + TRect1.length);
        System.out.println("Width of this rectangle is " + TRect1.width);
        TRect1.display();
    }
}
```



การห่อหุ้ม

```
public class Rectangle2{
    private double length;
    private double width;
    public Rectangle2(){
        this.length = 1;
        this.width = 1; }
    public Rectangle2(double l, double w){
        this.length =l;
        this.width = w; }
    public double getLength(){
        return length; }
    public double getWidth(){
        return width; }
    public double getArea(){
        return length*width; }
    public void display(){
        System.out.println("Area of this rectangle is " + getArea());
    }
}
```

Rectangle2

- length: double
- width: double

+ Rectangle2()
+ Rectangle2(l:double, w:double)
+ getLength() : double
+ getWidth() : double
+ getArea() : double
+ display() : void

การห่อหุ้ม

```
import java.util.Scanner;
public class TestRectangle2 {
    public static void main(String[] args) {
        Rectangle2 TRect = new Rectangle2();
        TRect.display();
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter Length: ");
        double l = scan.nextDouble();
        System.out.println("Enter Width: ");
        double w = scan.nextDouble();
        Rectangle2 TRect1 = new Rectangle2(l,w);
        System.out.println("Length of this rectangle is " + TRect1.getLength());
        System.out.println("Width of this rectangle is " + TRect1.getWidth());
        TRect1.display();
    }
}
```

